IoT — Extra Practice Bank

EXTRA PRACTICE · Weeks 1–6 · Mock paper 2 + 293 bank questions · Answers at the end

How to use this file.

Part	What	Questions
Mock 2	A second full paper in the teacher's format (Sections A–D, 100 marks)	18
A	Multiple choice, by week	155
B	True / False	50
C	Drills: duty cycle and energy, data volume, IP addresses, moving averages, MQTT wildcards, connectivity choice, layers, code reading	73
D	Short answer	15

Do Mock 2 under time pressure (about 90 minutes), then use Parts A–D to fix your weak weeks. All answers are in the key at the end.

Mock Paper 2 – IoT Fundamentals (Weeks 1–6)

Name: _____ **Student ID:** _____

Section A – Multiple choice (20 marks – 10 questions x 2 marks)

1. Which is the correct order of the physical–digital loop?

A. Action → Sense → Decision → Data → Process
B. Process → Data → Sense → Action → Decision
C. Sense → Data → Process → Decision → Action
D. Data → Action → Sense → Process → Decision

2. Which statement describes an IoT system rather than a traditional computing system?

A. Sensors generate the data, and the system may act automatically
B. A human types in all the data
C. Only a few computers are involved, in a purely digital environment
D. Every action is started by the user

3. One sensor produces one reading every second. How many readings does it produce per day?

A. 1,440
B. 3,600
C. 8,640
D. 86,400

4. Which connectivity requirement asks, "Can communication continue while the device moves?"

A. Range
B. Mobility
C. Data rate
D. Latency

5. A wearable heart-rate patch sends small readings to the user's smartphone and must run on a tiny battery. Which technology fits best?
   A. Ethernet
   B. LoRa
   C. BLE
   D. 5G

6. Which cellular LPWAN technology is best suited to small, **moving**, battery-powered trackers?
   A. LTE-M
   B. NB-IoT
   C. LoRa
   D. RFID

7. What is the correct order of the TCP three-way handshake?
   A. ACK, SYN, SYN-ACK
   B. SYN-ACK, SYN, ACK
   C. SYN, ACK, SYN-ACK
   D. SYN, SYN-ACK, ACK

8. A client sends `GET /sensors/99/temperature`, but sensor 99 does not exist. Which HTTP status code is expected?
   A. 200
   B. 404
   C. 201
   D. 500

9. Which statement about MQTT is correct?
   A. MQTT payloads must always be JSON
   B. MQTT typically runs over TCP and uses a broker to distribute messages by topic
   C. MQTT runs over UDP and needs no broker
   D. MQTT is a request–response protocol like HTTP

10. Which topic is matched by `factory/#`?
    A. `office/room101/temperature`
    B. `factories/machine01`
    C. `factory` (and every topic below it)
    D. Nothing, because `#` must be in the middle

Section B – Short Answer (30 marks – 5 questions x 6 marks)

11. The physical–digital loop

A smart irrigation system measures soil moisture and switches on a water pump when the soil is too dry. Describe each step of the physical–digital loop for this system, and identify which part is the **data flow** and which is the **control flow**.

12. TCP and UDP experiments

In the week 6 practical:

a. The TCP server is stopped and `tcp_client.py` is run again. What happens, and why? (2)

b. The UDP receiver is stopped and `udp_sender.py` is run again. What happens, and why? (2)

c. What do the two experiments show about the two transport protocols? (2)

13. Range, bandwidth and data rate

a. Explain the difference between bandwidth and data rate. (2)

b. A datasheet says a radio has "range = 100 m". Why might it fail at 60 m inside a factory? (2)

c. A temperature sensor sends 50 bytes every 5 minutes. Does it need a 1 Gbit/s link? Explain. (2)

14. MQTT wildcards

A broker has these topics:

```
farm/field1/soil/moisture
farm/field1/soil/temperature
farm/field2/soil/moisture
farm/field2/air/temperature
office/room1/air/temperature
```

List the topics received by each subscription:

a. `farm/+/soil/moisture` (2)

b. `farm/field1/#` (2)

c. `+/+/air/temperature` (2)

15. HTTP in IoT

A dashboard sends: `GET /api/sensors/7/temperature` with `Host: iot.example.com`.

a. Identify the action, the resource and the destination service. (3)

b. Which HTTP method would you use to **change** the reporting interval of sensor 7, and which to **submit** a new reading? (2)

c. What does status code 201 mean? (1)

Section C – Applied Scenario Questions (40 marks – 2 questions x 20 marks)

16. Electric Scooter Rental Fleet

A company rents 800 electric scooters across a whole city. Each scooter:

- reports its GPS location and battery level every 30 seconds while it is being ridden;
- moves through many different districts and streets;
- runs on its own battery;
- must be locked remotely by the company if a rental is not paid.

a. Identify three communication requirements for this fleet. (3 marks)

b. Choose a suitable connectivity technology and give two reasons. (5 marks)

c. Explain why Wi-Fi and LoRa are less suitable here. (3 marks)

d. Each report is sent as JSON. Explain why the company might use **compact JSON** (no extra spaces) and what this has to do with battery and network cost. (4 marks)

e. The scooters publish to an MQTT broker. Propose a topic structure for location and for lock commands, and explain how a **Last Will** message could help the operations team. (5 marks)

17. Greenhouse Edge Loop with Node-RED

A greenhouse uses the week 6 setup: a Python sensor publishes temperature to `greenhouse/zone1/temperature`; Node-RED decides whether to turn on a fan; a Python actuator script switches the fan.

The Node-RED flow is:

```
MQTT In → JSON → Switch → Change (FAN ON / FAN OFF) → MQTT Out
```

a. Explain the purpose of each of the five nodes. (5 marks)

b. Explain the difference between `msg.topic` and `msg.payload`, with an example from this flow. (2 marks)

c. In what order should you start the broker, the actuator and the sensor? Why does the order matter? (3 marks)

d. Explain the meaning of each part of: `mosquitto_pub -h localhost -t "greenhouse/zone1/fan/command" -m "FAN ON"` (4 marks)

e. A second dashboard is added that also subscribes to `greenhouse/zone1/temperature`. Does the sensor program need to change? Explain using the idea of **decoupling**. (6 marks)

Section D — Integrated IoT Communication (10 marks)

18 — Trace the Data (constrained path)

```
Room temperature sensor
↓
CoAP  (GET /temperature)
↓
UDP
↓
IPv6
↓
6LoWPAN
↓
IEEE 802.15.4
↓
Border router
↓
IPv6 Internet
↓
Building management application
```

Explain the role of each of the following:

Technology	Role
CoAP	
UDP	
IPv6	
6LoWPAN	
IEEE 802.15.4	
Border router	

Why is this path used instead of Wi-Fi → IP → TCP → HTTP for this sensor?

Part A — Multiple Choice

Week 1 — Introduction to IoT

1. IoT connects:

A. Only computers to printers
B. The physical world with the digital world
C. Only phones to the Internet
D. Only factories to banks

2. Which question is one of the four questions IoT data should answer?

A. What colour is the device?
B. Who manufactured the sensor?
C. How expensive is the cloud?
D. Is the condition normal or abnormal?

3. Which component **transmits** data from sensors to the processing system?

A. Network
B. Physical object
C. Application
D. Actuator

4. Which component makes information available to the user and enables control?

A. Sensor
B. Physical object
C. Application
D. Network

5. What does an accelerometer measure?

A. Rotation
B. Linear motion
C. Location
D. Brightness

6. What does a gyroscope measure?

A. Rotation and angular velocity
B. Heat
C. Moisture
D. Location

7. What does GPS provide?

A. Heat
B. Image
C. Brightness
D. Location

8. What does a typical IoT device contain?

A. Only a sensor
B. Only a battery and a screen
C. Sensor, processor, memory, communication module, power source and software
D. A keyboard and a monitor

9. Network selection in IoT depends on:

A. Distance, speed, power consumption, cost and reliability
B. The colour of the device
C. The user's age
D. The brand of phone

10. In a smart home, what does the **gateway** do?

A. Measures temperature
B. Provides the connection between devices and external services
C. Stores one year of history
D. Replaces the cloud

11. What is **predictive maintenance**?

A. Repairing a machine only after it breaks
B. Replacing all machines every year
C. Turning machines off at night
D. Detecting anomalies and predicting failure before it happens

12. A device becomes part of IoT when it can:

A. Show a colourful screen
B. Run on mains power
C. Sense or generate data, communicate through a network, and interact with another digital system
D. Be bought online

13. Which application priority belongs to a **smart city**?

A. Scalability
B. Convenience and cost
C. Fast response and reliability
D. Long range and low power

14. Which priorities belong to **healthcare**?

A. Convenience and cost
B. Reliability and privacy
C. Long range and low power
D. Scalability only

15. Which priorities belong to **smart agriculture**?

A. Convenience and cost
B. Reliability and privacy
C. Fast response
D. Long range and low power

16. 1,000 sensors each produce one reading per second. How many readings per day?

A. 86,400
B. 8,640,000
C. 86,400,000
D. 1,000,000

17. What does **heterogeneity** mean in IoT?

A. All devices are identical

B. Devices differ in manufacturers, sensors, operating systems, data formats and communication technologies

C. Every device uses Wi-Fi

D. All data is stored in one place

18. What is **scalability**?

A. Whether the system keeps working efficiently as the number of devices or the amount of data grows

B. The weight of a device

C. The colour depth of a camera

D. The price of one sensor

19. What is the main energy lesson of week 1?

A. Sensors use no energy

B. Processing uses all the energy

C. Batteries last forever

D. Sending data can consume significant energy

20. What is the response to the **latency** challenge in safety-critical applications?

A. Send everything to the cloud

B. Buy a bigger battery

C. Use faster local processing

D. Collect more data

21. What is the response to the **bandwidth** challenge?

A. Filter locally and send only important data to the cloud

B. Send every raw reading

C. Add more sensors

D. Turn off the network

22. What is the response to **network dependency** (the Internet fails)?

A. Stop the whole system

B. Local processing, local storage and retry mechanisms

C. Delete the data

D. Use a longer cable

23. What is the reliability response to a **gateway failure**?

A. Health check of the sensor

B. Power saving

C. Later synchronization only

D. A backup gateway or local operation

24. Which question about **privacy** leads to edge computing?

A. How fast is the processor?

B. How many sensors are there?

C. Does all collected data need to leave the local environment?

D. What is the price of storage?

25. What is the order of the smart health monitoring system in Activity 4?

A. Doctor → Cloud → Patient → Sensor → Internet → Phone
B. Patient → Wearable sensor → Smartphone → Internet → Cloud platform → Doctor dashboard
C. Smartphone → Patient → Doctor → Sensor → Cloud → Internet
D. Cloud → Internet → Patient → Doctor → Phone → Sensor

Weeks 2–3 — Sensor Data, Moving Averages and Data Representation

26. What is a **virtual sensor** in the week 2 practical?

A. A real thermometer
B. A camera
C. A program that produces simulated readings, for example with `random.uniform()`
D. A cloud dashboard

27. What is **time-series data**?

A. Values recorded with timestamps over time
B. A single value without time
C. A list of IP addresses
D. An MQTT topic

28. Why is the timestamp needed?

A. It makes the payload smaller
B. It encrypts the data
C. It replaces the device ID
D. It shows when the value was measured, how fast it changes and whether it is current

29. `time.sleep(1)` inside the reading loop sets:

A. The sensor accuracy
B. The sampling interval: one reading per second
C. The battery level
D. The moving-average window

30. What is the purpose of a moving average?

A. To delete old data
B. To make the sensor faster
C. To smooth noise and short-term fluctuations
D. To encrypt readings

31. Readings 20, 22 and 30. What is the moving average with window 3?

A. 24
B. 22
C. 26
D. 72

32. Why does the code print "Waiting" instead of 0 before three readings exist?

A. 0 is not allowed in Python
B. The sensor is broken
C. The program crashed
D. The average is not available yet, and 0 would be false data

33. In the moving-average code, `recent_readings.pop(0)` removes:

A. The newest reading
B. The oldest reading
C. All readings
D. The average

34. What happens with a **larger** moving-average window?

A. Faster response, more noise
B. Exactly the same line
C. Smoother line, but slower response to real changes
D. No readings needed

35. A missing measurement at 10:02 is an example of:

A. A data-quality problem
B. An IPv6 address
C. A Wi-Fi connection
D. An MQTT subscription

36. What does **validation** check in the cold-chain assignment?

A. Whether the value is good for the medicine
B. Whether the value is physically possible
C. Whether the battery is new
D. Whether the JSON is compact

37. What does **status** check in the cold-chain assignment?

A. Whether the value is physically possible
B. Whether the timestamp exists
C. Whether the payload is bytes
D. Whether the value is acceptable for the cargo and situation

38. Humidity 75 % (normal 30–70 %). Status?

A. NORMAL
B. INVALID
C. HUMIDITY_WARNING
D. TOO_WARM

39. Battery 15 % (low battery below 20 %). Status?

A. LOW_BATTERY
B. NORMAL
C. INVALID
D. TOO_COLD

40. Temperature 1.5 °C (normal 2–8 °C). Status?

A. NORMAL
B. TOO_COLD
C. TOO_WARM
D. INVALID

41. What does `record["status"]["temperature"]` return?

A. The whole record
B. The device ID
C. The temperature value
D. The temperature status, such as `"NORMAL"`

42. What is **serialization**?

A. Increasing wireless range
B. Selecting an IP address
C. Converting structured data into a representation suitable for transmission or storage
D. Increasing sensor accuracy

43. What does `type()` show before and after `json.dumps(record)`?

A. `dict` before, `str` after
B. `str` before, `dict` after
C. `list` before, `bytes` after
D. `dict` before and after

44. Why encode the JSON string with UTF-8 to measure its size?

A. To encrypt it
B. Because the payload size in bytes is what is actually transmitted
C. To make it a dictionary again
D. To add a timestamp

45. What does compact JSON do?

A. Adds indentation for humans
B. Removes field names
C. Changes the values
D. Removes unnecessary spaces, so the payload is smaller

46. Why is a JSON payload larger than the raw numbers it carries?

A. JSON compresses data
B. JSON is binary
C. JSON carries field names, quotes and structure as text
D. JSON adds encryption

47. What does `json.loads()` do?

A. Converts JSON text into a Python dictionary
B. Converts a dictionary into JSON text
C. Sends data to the broker
D. Opens a socket

48. What is the difference between a Python dictionary and a JSON string?

A. They are the same
B. A dictionary is an in-memory data structure; JSON is a text format other systems can read
C. JSON can only be used in Python
D. A dictionary can be sent directly over a socket

49. Why does a smaller payload matter for an IoT device?

A. It changes the sensor value
B. It makes the device heavier
C. It is required by IPv4
D. Less data means less radio-on time, energy and bandwidth

50. In the week 2 humidity assignment, which statistics were printed for the 100 readings?

A. Only the last reading
B. Only the first reading
C. Average, minimum and maximum
D. The IP address

Week 4 — IoT Connectivity Technologies

51. What is **IoT connectivity**?

A. A wired or wireless link that lets a device exchange data with another device, gateway or network
B. The rules for application messages
C. A JSON format
D. A cloud database

52. Connectivity versus communication protocol, in the lecture's analogy:

A. Connectivity = rules of the road; protocol = the road
B. Both are the road
C. Connectivity = the road; protocol = the rules for how information travels on it
D. Both are the rules

53. Which are the four common connectivity requirements?

A. Colour, size, weight, price
B. Range, data rate, power, mobility
C. CPU, RAM, disk, screen
D. Topic, payload, QoS, broker

54. What is **range**?

A. The amount of data per second
B. The battery life
C. The delay of a message
D. The maximum practical communication distance between nodes

55. Real range depends on:

A. Transmitter power, antenna, frequency, receiver sensitivity, obstacles and interference
B. Only the datasheet number
C. The colour of the device
D. The JSON payload

56. In which unit is data rate typically measured?

A. Metres
B. Milliamps
C. Bits per second
D. Degrees

57. A 1 Gbit/s link is about:

A. 1 GB/s
B. 125 MB/s
C. 8 MB/s
D. 1 MB/s

58. Which radio state uses the **least** power?

A. Transmit
B. Receive
C. Idle / listening
D. Sleep

59. A radio transmits 1 second every minute. Duty cycle?

A. 1.67 %
B. 0.028 %
C. 16.7 %
D. 60 %

60. A radio transmits 1 second every hour. Duty cycle?

A. 1.67 %
B. 2.8 %
C. 0.028 %
D. 0.28 %

61. Device A transmits 1 s every minute; device B 1 s every hour. A uses about how much more transmit energy?

A. The same
B. 60 times
C. 24 times
D. 3,600 times

62. Which formula estimates transmit energy?

A. $E \approx V \div t$
B. $E \approx I \div t$
C. $E \approx t \div I$
D. $E \approx I \times t$

63. An emergency gas-leak sensor must prioritize:

A. Sending rarely to save battery
B. Large video data
C. Low latency and a rapid alarm
D. Long-term storage

64. What is **latency**?

A. The delay between sending information and its arrival or use at the destination
B. The maximum distance
C. The battery size
D. The number of devices

65. What is **mobility**?

A. The weight of the device
B. How far the link can reach
C. How many devices share the network
D. Whether the device changes location while operating and communication must continue

66. A smartwatch has which mobility level?

A. Fixed
B. Local mobility
C. Wide-area mobility
D. None

67. Range versus mobility:

A. Range = how far the link reaches; mobility = whether communication continues while moving
B. They are the same
C. Range = moving; mobility = distance
D. Neither matters

68. Concrete, steel and machinery affecting a signal belong to which selection factor?

A. Device density
B. Cost
C. Environment
D. Data rate

69. The connectivity trade-off triangle says you rarely get all three of:

A. Topic, payload, QoS
B. Cost, colour, size
C. IP, TCP, MQTT
D. Long range, high data rate and very low power

70. What is an advantage of **wired** connectivity?

A. Mobility
B. A stable link with less radio-frequency interference
C. No installation needed
D. Flexible deployment

71. What does **PoE** provide?

A. Data and electrical power over one Ethernet cable
B. Wireless power
C. A cellular connection
D. Encryption

72. Why is Wi-Fi a poor choice for a coin-cell sensor that sends tiny data?

A. Wi-Fi cannot send numbers
B. Wi-Fi has no access points
C. Its relatively high power drains the small battery
D. Wi-Fi has too little range for a room

73. Why is BLE low power?

A. It uses a cable
B. It sends video
C. It is always transmitting
D. The radio stays off most of the time and wakes briefly to send small data

74. What infrastructure does BLE typically use?

A. A cellular tower
B. A phone or gateway
C. A satellite
D. An Ethernet switch

75. What is **IEEE 802.15.4**?

A. A low-rate, low-power wireless foundation used by technologies such as Zigbee
B. A cellular standard
C. An application protocol
D. A JSON library

76. What is the main limitation of a **star** topology?

A. Too many alternative paths
B. Relay burden
C. Central point dependency
D. No central node

77. Why does a **mesh** topology cost more energy?

A. It has no radio
B. Nodes relay other nodes' data as well as their own
C. It uses cables
D. It never sleeps by law

78. What is a **point-to-point** topology?

A. Many devices through one central node
B. Devices relaying through neighbours
C. A satellite network
D. Two endpoints communicating directly

79. What is **line of sight**?

A. An unobstructed straight path between the transmitting and receiving antennas
B. A type of cable
C. A QR code
D. A cellular tower

80. What is the range of **NFC**?

A. Kilometres
B. Hundreds of metres
C. Centimetres
D. Tens of metres

81. What is the trade-off of **LPWAN**?

A. Very high power
B. Very short range
C. No battery
D. Very low data rate

82. Cellular LPWAN (NB-IoT, LTE-M) uses:

A. Unlicensed open bands with private gateways
B. Licensed spectrum run by telecom operators
C. Cables
D. Optical reading

83. What is the typical application of **NB-IoT**?

A. Smart meters: static, low data, good indoor coverage
B. Video streaming
C. Fast-moving trackers
D. Contactless payment

84. LoRa battery consumption compared with cellular:

A. Higher than a smartphone
B. Exactly the same
C. Ultra-low: a device can last up to about 10 years on one battery
D. It needs mains power

85. Why is LTE-M **not** chosen for a wearable that talks to a nearby phone?

A. LTE-M has no coverage
B. Greater complexity (modem, SIM) and higher energy and service cost
C. LTE-M is a wired technology
D. LTE-M cannot send numbers

Week 5 — Communication Protocols and Messaging

86. In the package-delivery analogy, the IP address is like:

A. The room number
B. The rules for the content
C. The university and building: which host
D. The courier's name

87. The transport layer answers:

A. Which application should receive the data, and how should it be delivered?
B. How does the device access the network?
C. What does the data mean?
D. Which building is it?

88. When is traffic sent to the **default gateway**?

A. Always
B. When the destination is on the same local network
C. Never
D. When the destination is outside the local network

89. What is true about a **private** IP address?

A. It is globally routable
B. It is used inside local networks and is not directly routable on the public Internet
C. It is always IPv6
D. It identifies a person

90. What does **NAT** do?

A. Encrypts traffic
B. Compresses headers
C. Translates between private and external addresses at the network boundary, conserving public IPv4 addresses
D. Chooses MQTT topics

91. How many bits is an IPv4 address?

A. 32
B. 64
C. 128
D. 16

92. How many bits is an IPv6 address?

A. 32
B. 48
C. 64
D. 128

93. What is `2001:0db8:0000:0000:0000:ff00:0042:8329` in shortened form?

A. `2001:db8:ff00:42:8329`

B. `2001:db8::ff00:42:8329`

C. `2001::db8::ff00::42:8329`

D. `21:db8::ff:42:8329`

94. 6LoWPAN header compression reduces the IPv6 header from:

A. 20 bytes to 10 bytes

B. 128 bytes to 32 bytes

C. 40 bytes to as few as 2–7 bytes

D. 8 bytes to 1 byte

95. What does the **border router** do in a 6LoWPAN network?

A. Connects the low-power mesh to the IPv6 Internet

B. Measures temperature

C. Encrypts JSON

D. Runs the MQTT broker

96. Port **1883** is used by:

A. HTTP

B. A database

C. SSH

D. MQTT

97. Port **80** is used by:

A. MQTT

B. HTTP

C. CoAP

D. DNS

98. Which pair is correct?

A. Network layer = host-to-host; transport layer = process-to-process

B. Network layer = process-to-process; transport = host-to-host

C. Both are host-to-host

D. Both are process-to-process

99. Which are functions of the transport layer?

A. Header compression and mesh routing

B. Topic routing

C. Application multiplexing through ports, segmentation, and delivery behavior

D. JSON parsing

100. Why does TCP perform a handshake?

A. To compress data

B. To find the IP address

C. To choose a topic

D. To establish communication state and coordinate sequence numbers

101. In TCP, the ACK number is:

A. The first byte sent

B. The next expected byte

C. The port number

D. The QoS level

102. If the expected TCP acknowledgment does not arrive:

A. The sender retransmits
B. The data is ignored forever
C. The connection becomes UDP
D. The broker stores it

103. Which is true about UDP flow control?

A. It slows down when the receiver is busy
B. It waits for ACKs
C. There is none: UDP sends as fast as the application produces data
D. It uses a handshake

104. Why is an application protocol needed on top of TCP?

A. TCP cannot carry bytes
B. TCP has no ports
C. TCP is too slow
D. TCP delivers bits but does not know whether they are a request, a command or a temperature

105. What is **resource-oriented** communication?

A. Sending everything to one topic
B. Organizing data and functions as uniquely addressable paths (URIs)
C. Using only UDP
D. Polling every second

106. HTTP runs over:

A. TCP
B. UDP only
C. 6LoWPAN only
D. MQTT

107. In IoT, `GET` means:

A. Delete a configuration
B. Update a configuration
C. Read sensor status
D. Submit a new reading

108. `PUT` means:

A. Read status
B. Delete
C. Submit a new reading
D. Update a resource or configuration

109. `POST` means:

A. Read status
B. Submit data or create an action
C. Delete a resource
D. Ping the server

110. Status code **201** means:

A. Created
B. OK
C. Not found
D. Server error

111. Status code **500** means:

A. OK
B. Bad request
C. Server error
D. Created

112. Status code **400** means:

A. OK
B. Created
C. Server error
D. Bad request

113. CoAP commonly uses which transport?

A. TCP
B. UDP
C. Ethernet
D. MQTT

114. CoAP messages are:

A. Compact binary
B. Long text like HTTP
C. Always JSON
D. Video streams

115. What is **polling**?

A. Publishing to a topic
B. Subscribing with a wildcard
C. Repeatedly asking a device whether it has new data, even when nothing has changed
D. Compressing a header

116. What are the three roles in publish–subscribe?

A. Client, server, database
B. Sender, router, switch
C. Sensor, gateway, cloud
D. Publisher, broker, subscriber

117. What is the benefit of **decoupling** through a broker?

A. The sensor must know every destination
B. The sensor talks mainly to the broker, and adding new applications is simpler
C. Every application must poll the sensor
D. It removes the need for a network

118. What is the MQTT connection sequence?

A. TCP connection → CONNECT → CONNACK → PUBLISH / SUBSCRIBE
B. PUBLISH → CONNECT → TCP
C. CONNACK → CONNECT → UDP
D. SUBSCRIBE → TCP → CONNECT

119. Which is a **good** MQTT topic?

A. `Sensor1`
B. `Data3`
C. `factory/line1/machine01/temperature`
D. `x`

120. What does `#` match?

A. Exactly one level
B. Only the first level
C. Nothing
D. Zero or more remaining levels

121. Why does `factory/+/temperature` not match `factory/line1/machine01/temperature`?

A. `+` matches all levels
B. `+` matches exactly one level, and that topic has an extra level
C. Topics cannot contain numbers
D. Wildcards only work with QoS 2

122. 500 machines each publish temperature. With `factory/+/temperature`, how many subscriptions does the dashboard need?

A. 1
B. 500
C. 1,500
D. 3

123. What is a **retained message**?

A. The full history of a topic
B. A message published when a client crashes
C. The latest message on a topic, delivered immediately to new subscribers
D. A message with QoS 2

124. What is a **Last Will** message?

A. The first message after connecting
B. A retained history
C. A QoS 0 telemetry message
D. A message defined at connection time and published only if the client disconnects unexpectedly

125. Which stack fits a battery-powered 802.15.4 sensor whose temperature is requested?

A. Wi-Fi → IP → TCP → MQTT
B. IEEE 802.15.4 → 6LoWPAN → IPv6 → UDP → CoAP
C. Ethernet → IPv4 → TCP → HTTP
D. BLE → MQTT → JSON → TCP

Week 6 — Practical: From Communication to Messaging

126. What is `127.0.0.1`?

A. A public IP address
B. The loopback address: this same computer (localhost)
C. The MQTT port
D. A MAC address

127. What is a **socket**?

A. A JSON file
B. An MQTT topic
C. A physical cable
D. The programming interface to transport communication

128. `AF_INET6` selects:

A. IPv6 addressing
B. IPv4 addressing
C. TCP
D. UDP

129. `SOCK_STREAM` selects:

A. UDP
B. IPv6
C. TCP
D. MQTT

130. What does `bind()` do on the server?

A. Sends data
B. Reserves the server endpoint (IP and port)
C. Closes the socket
D. Parses JSON

131. What does `listen()` do?

A. Makes the program wait for incoming TCP clients
B. Sends a datagram
C. Publishes to a topic
D. Decodes bytes

132. What does `accept()` do?

A. Sends a response
B. Converts bytes to text
C. Creates a UDP socket
D. Blocks until a client connects

133. What does `recv(1024)` do?

A. Sends 1024 bytes
B. Opens port 1024
C. Receives up to 1024 bytes
D. Waits 1024 seconds

134. What does `sendall()` do?

A. Sends to every computer
B. Sends all the bytes over the connection
C. Publishes to all topics
D. Sends a datagram

135. What does `recvfrom()` return?

A. The data and the sender's address
B. Only the port number
C. A dictionary
D. A TCP connection

136. Which calls are **not** used for UDP?

A. `sendto()` and `recvfrom()`
B. `bind()`
C. `socket()`
D. `connect()` and `accept()`

137. Over TCP, the client sends "Hello" then "World". The server may read:

A. Two separate datagrams
B. Nothing
C. "HelloWorld", because TCP does not preserve message boundaries
D. "World" then "Hello" always

138. Over UDP, the sender sends "Message A" then "Message B". The receiver gets:

A. "Message AMessage B" in one piece
B. Two separate datagrams (if they arrive)
C. A TCP stream
D. Nothing ever

139. Running `tcp_client.py` while no server is listening gives:

A. A connection error, because TCP is connection-oriented
B. "Message sent"
C. A JSON error
D. A retained message

140. Which analogy does week 6 use?

A. TCP = postcard; UDP = courier
B. Both are postcards
C. Both are couriers
D. TCP = sealed courier delivery; UDP = postcard

141. How are MQTT and TCP related?

A. MQTT replaces TCP
B. They are the same
C. MQTT typically runs over TCP and adds application-level messaging
D. MQTT runs over UDP only

142. In `mqtt_subscriber.py`, what happens inside `on_connect()`?

A. The message is printed
B. `client.subscribe(TOPIC)` asks the broker for messages on the topic
C. The broker starts
D. The JSON is parsed

143. When is `on_message()` called?

A. Automatically, whenever a message on a matching topic arrives
B. Only once at startup
C. When the client disconnects
D. Every second, even without messages

144. What does `loop_start()` do in `mqtt_sensor.py`?

A. Starts the broker
B. Starts an infinite `for` loop
C. Subscribes to all topics
D. Runs MQTT network processing in the background

145. What does `keepalive = 60` mean?

A. The sensor publishes every 60 seconds
B. The broker deletes messages after 60 seconds
C. If there is no normal traffic for up to 60 seconds, the client still sends a keep-alive so the connection stays active
D. The temperature limit is 60 °C

146. `qos=0` in `mqtt_sensor.py` means:

A. Exactly once
B. No MQTT-level acknowledgment or redelivery
C. At least once
D. The message is retained

147. What does the MQTT publisher **not** need to know?

A. The subscribers' IP addresses
B. The broker address
C. The topic
D. The payload

148. What does `mosquitto -v` do?

A. Publishes a message
B. Subscribes to a topic
C. Shows the version only and exits
D. Starts the broker and prints connection and message activity

149. In `mosquitto_pub`, the option `-t` sets:

A. The host
B. The message
C. The topic
D. The time

150. `mosquitto_pub -h localhost -t "test/topic" -m "Hello IoT"` is run while a verbose subscriber is listening. What does the subscriber print?

A. `Hello IoT test/topic`
B. `test/topic Hello IoT`
C. `localhost`
D. Nothing

151. What rule does the week 6 edge processor apply?

A. Temperature > 30 °C → publish `FAN ON`, otherwise `FAN OFF`
B. Temperature < 30 °C → `FAN ON`
C. Always `FAN ON`
D. Humidity > 30 % → `FAN OFF`

152. To which topic does the edge processor publish its command?

A. `factory/machine01/temperature`
B. `test/topic`
C. `factory/#`
D. `factory/machine01/fan/command`

153. In the actuator, why is `message.payload.decode()` used?

A. To encrypt the command
B. To publish again
C. The payload arrives as bytes and must be converted to text
D. To compress the message

154. Where does Node-RED run after starting it?

A. `127.0.0.1:1883`
B. `http://127.0.0.1:1880`
C. `127.0.0.1:5000`
D. `127.0.0.1:80`

155. What does the Node-RED **Switch** node do in the demo?

A. Tests the threshold `msg.payload.temperature > 30`
B. Subscribes to the sensor topic
C. Publishes the command
D. Parses JSON

Part B — True / False

Write **T** or **F**. If false, correct the statement.

1. IoT is only about connecting devices to the Internet.
2. A complete IoT system includes device, data, communication, processing, application and user or action.
3. Storing one year of history is a typical device-level task.
4. More communication means more energy use and shorter battery life.
5. IoT systems can both observe and control the physical world.
6. Heterogeneity means all devices use the same data format.
7. A moving average with window 3 is available from the first reading.
8. A valid reading is always a normal reading.
9. `json.dumps()` converts a JSON string into a dictionary.
10. Compact JSON produces a smaller payload than indented JSON.
11. "Range = 100 m" means the link always works at 100 m.
12. Bandwidth and data rate mean the same thing.
13. The sleep state of a radio uses very low power.
14. A lower duty cycle gives a longer battery life.
15. Wi-Fi is the best choice for every IoT device.
16. BLE is designed for short-range, battery-powered devices.
17. Mesh nodes relay other nodes' data, which costs extra energy.
18. RFID requires line of sight.
19. Passive RFID tags can work without a battery.
20. LoRa is a cellular technology run by telecom operators.
21. LoRa allows a private gateway.
22. LTE-M handles moving devices better than NB-IoT.
23. A single technology can maximize long range, high data rate and very low power at once.
24. PoE delivers data and power over one Ethernet cable.
25. IP finds the device; TCP/UDP deliver the data to the correct application.
26. Private IP addresses are globally routable.
27. IPv6 solves the battery and memory limits of constrained devices.
28. 6LoWPAN is an adaptation layer between IPv6 and constrained links.
29. The transport layer provides host-to-host communication.
30. UDP guarantees that packets arrive in order.
31. TCP can detect and ignore duplicate data.
32. HTTP uses the publish–subscribe model.

33. CoAP commonly runs over UDP.
34. HTTP status 404 means server error.
35. Polling scales well when there are many sensors.
36. An MQTT publisher must know the IP addresses of its subscribers.
37. MQTT requires JSON payloads.
38. The `+` wildcard matches several levels.
39. `factory/#` also matches the topic `factory`.
40. QoS 1 can deliver duplicates.
41. QoS 2 uses only PUBLISH and PUBACK.
42. A retained message stores the complete history of a topic.
43. A Last Will is published when a client disconnects normally.
44. HTTP, CoAP and MQTT can be used together in one IoT system.
45. A UDP socket must call `connect()` before sending.
46. A socket transmits bytes, not strings.
47. `recvfrom()` returns the data and the sender's address.
48. MQTT replaces TCP.
49. Start the subscriber after the publisher so that it receives the earlier messages.
50. The virtual actuator subscribes to the sensor topic.

Part C — Drills

C1. Duty cycle and energy

Calculate the duty cycle (1–5) or the transmit energy per day (6–9). Use $E \approx I \times t$.

1. Transmit 1 s every 30 s.
2. Transmit 0.5 s every 60 s.
3. Transmit 2 s every 10 minutes.
4. Transmit 1 s every 15 minutes.
5. Transmit 5 s every hour.
6. Transmit 1 s every minute at 20 mA.
7. Transmit 1 s every 5 minutes at 30 mA.
8. Transmit 3 s every hour at 50 mA.
9. Transmit 2 s every 10 minutes at 45 mA.
10. A 2400 mAh battery powers device A from the lecture (12 mAh/day transmit energy). How many days would the battery last if only transmitting counted? What if it transmitted once per hour instead (0.20 mAh/day)? Why is the real lifetime shorter?

C2. Data volume

1. One sensor, one reading every 10 seconds. Readings per day?

2. 50 sensors, one reading per minute each. Readings per day?
3. 1,000 sensors, one reading per second each. Readings per day?
4. 200 sensors, one 60-byte message per minute each. Messages and bytes per day?
5. 100 sensors read once per second. The edge sends only one average per sensor per minute to the cloud. How many messages per day before and after? By what factor is traffic reduced?

C3. IP addresses

1. Write `192.168.1.25` in binary.
2. Write `10.0.0.1` in binary.
3. Write `172.16.5.200` in binary.
4. Shorten `2001:0db8:0000:0000:0000:0000:0000:0001`.
5. Shorten `fe80:0000:0000:0000:0204:61ff:fe9d:f156`.
6. Shorten `2001:0db8:abcd:0012:0000:0000:0000:0025`.
7. Expand `2001:db8::ff00:42:8329` to all 8 groups.
8. A sensor `192.168.1.25` in network `192.168.1.0/24` sends to `192.168.1.50` and to `203.0.113.10`. Which message must go through the default gateway?

C4. Moving averages (window 3 unless stated)

1. Readings 22, 24, 29, 23. Give the output for each reading.
2. Readings 30, 30, 30, 60, 30. Give the output for each reading. What does the 60 look like after averaging?
3. Readings 10, 20, 30, 40, 50.
4. Readings 30, 30, 30, 60, 30 with **window 5**: what is the only available average? Which window reduces the spike more?

C5. MQTT wildcard matching

Broker topics:

Label	Topic
T1	`farm/field1/soil/moisture`
T2	`farm/field1/soil/temperature`
T3	`farm/field2/soil/moisture`
T4	`farm/field2/air/temperature`
T5	`farm/greenhouse/air/humidity`
T6	`office/room1/air/temperature`
T7	`farm`

Which topics (T1–T7) does each subscription receive?

1. `farm/+/soil/moisture`
2. `farm/field1/#`
3. `farm/+/+/temperature`

4. `farm/#`
5. `+/+/air/temperature`
6. `farm/+/air/+`
7. `farm/field1/soil/moisture`
8. `farm/+/moisture`
9. `#`
10. `+/room1/#`
11. `farm/greenhouse/+`
12. `farm/+`

C6. Choose the connectivity

Pick the best technology and give one reason.

1. A CCTV camera in a shop where cabling already exists.
2. A fitness band that syncs with a phone.
3. Vineyard soil sensors 5 km from the office, on batteries.
4. A water meter in a basement, anywhere in the city.
5. A shipping-container tracker crossing the country.
6. Library book inventory.
7. A contactless transit card.
8. A smart TV streaming video at home.
9. 200 low-power temperature sensors spread through an office building.
10. A remote ocean buoy far from any cellular network.
11. A factory robot arm with time-sensitive control, fixed in place.
12. A pet tracker that moves around a city.

C7. Which layer?

Write: connectivity/link, adaptation, network, transport, application, or data representation.

1. Wi-Fi
2. IPv6
3. UDP
4. MQTT
5. 6LoWPAN
6. Port number 1883
7. JSON
8. `AF_INET`
9. `SOCK_DGRAM`
10. CoAP

11. Ethernet
12. HTTP `GET`

C8. Code reading

1. What does `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` create?
2. What does `client.sendto(b"Hello", ("127.0.0.1", 5000))` do?
3. `client.send("Hello")` (a string, not bytes) fails. Why, and what is the fix?
4. What does `json.dumps({"t": 22.5})` return, and what is its type?
5. What does `len(json.dumps(record).encode("utf-8"))` measure?
6. `data, addr = sock.recvfrom(1024)`: what are `data` and `addr`?
7. The edge rule is `if temperature > 30: FAN ON else: FAN OFF`. The reading is exactly 30.0. Which command is published?
8. What does `message.payload.decode()` do?
9. `client.publish(TOPIC, payload, qos=1)`: which guarantee, and which acknowledgment message?
10. A subscriber runs `mosquitto_sub -h localhost -t "lab/#" -v`. Then `mosquitto_pub -h localhost -t "lab/room1/temp" -m "21"` is run. What does the subscriber print?

Part D — Short Answer

1. Using a smartwatch, explain why a single smart device is not a complete IoT system.
2. Name the six IoT challenges from week 1 and give one response to each.
3. Give five differences between a traditional computing system and an IoT system.
4. Why does an IoT record need a timestamp, and why keep previous readings?
5. Explain validation versus status using the cold-chain monitor.
6. Explain the difference between a Python dictionary and a JSON string, what serialization is, and why payload size matters.
7. Explain the difference between connectivity and a communication protocol.
8. Explain duty cycle and why a low duty cycle extends battery life.
9. Compare Wi-Fi, BLE, LoRa, NB-IoT and LTE-M with one typical use each.
10. Compare star and mesh topologies.
11. Explain the four ideas in TCP and the four characteristics of UDP.
12. Compare HTTP and CoAP.
13. Explain the polling problem and how publish–subscribe solves it.
14. Compare a retained message and a Last Will message.
15. Describe the complete week 6 edge loop: which programs run, which topics they use, and how it shows sense → communicate → process → act.

Answer Key

Mock Paper 2

Section A: 1 C · **2** A · **3** D · **4** B · **5** C · **6** A · **7** D · **8** B · **9** B · **10** C

11. *Sense*: the soil-moisture sensor measures the soil in the physical environment. *Data*: the reading becomes a value with device ID, time and unit (e.g. 18 %). *Process*: the system compares it with a threshold (or a moving average). *Decision*: "the soil is too dry". *Action*: the pump switches on, water raises the moisture, and the loop starts again. **Data flow** = sensor → network → processing/application → user; **control flow** = decision/user → application → network → pump (actuator).

12. a. The client's `connect()` **fails with a connection error**: TCP is connection-oriented and cannot exchange application data unless a listening server accepts the connection. b. The sender **still prints "UDP message sent"**, but nothing is received: UDP has no connection setup and no built-in delivery confirmation. c. TCP = connection-oriented, reliable, ordered, knows if the other side is missing; UDP = connectionless, lightweight, no guarantees. Same IP + port addressing, different delivery behavior.

13. a. **Bandwidth** = the **maximum** amount that can be transmitted in a given time; **data rate** = the rate **actually** transferred. More bandwidth can support a higher data rate, but they are not the same. b. Real range depends on transmitter power, antenna, frequency, receiver sensitivity, obstacles and interference: walls, steel (reflects/blocks) and machinery (electromagnetic interference) shorten it. 100 m in an open field ≠ 100 m in a factory. c. No: 50 bytes every 5 minutes is a tiny data rate. A low-rate, low-power technology is better; a high-rate link wastes power and cost.

14. a. `farm/field1/soil/moisture`, `farm/field2/soil/moisture`. b. `farm/field1/soil/moisture`, `farm/field1/soil/temperature`. c. `farm/field2/air/temperature`, `office/room1/air/temperature`.

15. a. Action = `GET` (retrieve), resource = `/api/sensors/7/temperature`, destination service = `Host: iot.example.com`. b. Change interval → **PUT** (update a resource/configuration); submit a reading → **POST**. c. **201 Created**: a new resource was created.

16. a. Any three: **wide-area coverage** across the city; **mobility** (communication must continue while moving between districts); **two-way communication** (remote lock commands); low-to-moderate **data rate** (location every 30 s); **power efficiency** on the scooter battery; reliability for lock commands. b. **LTE-M (cellular IoT)**: wide coverage using the operator's existing infrastructure, so no private gateways are needed; **handles moving devices efficiently**; higher data rate than NB-IoT, enough for frequent location reports; suits small, mobile, battery-powered devices. c. **Wi-Fi**: limited local range; the scooter loses connection as soon as it leaves an access point's coverage. **LoRa**: very low data rate and limited mobility; frequent location reports and timely lock commands to 800 moving scooters need wider, mobile coverage. d. Compact JSON removes unnecessary spaces, so each message has **fewer bytes**. Fewer bytes → shorter transmission → **less radio-on time** → **less energy**; with 800 scooters reporting every 30 s, smaller payloads also reduce **network traffic and data cost**. The information does not change. e. Topics e.g. `fleet/scooter/<id>/location` (published by each scooter) and `fleet/scooter/<id>/command` (subscribed by each scooter; the company publishes `LOCK`). A dashboard can subscribe to `fleet/scooter/+/location`. **Last Will**: each scooter sets, when connecting, a will such as topic `fleet/scooter/<id>/status`, payload `offline`; if the scooter crashes or loses its connection unexpectedly, the broker publishes it, so operations immediately know which scooter is unreachable.

17. a. **MQTT In**: subscribes to `greenhouse/zone1/temperature`. **JSON**: parses the JSON text so `msg.payload` becomes an object. **Switch**: tests `msg.payload.temperature > 30` (threshold decision). **Change**: sets `msg.payload` to `FAN ON` or `FAN OFF`. **MQTT Out**: publishes the command to the fan command topic. b. `msg.topic` = the label/route, e.g. `greenhouse/zone1/temperature`; `msg.payload` = the content, e.g. `{"temperature": 34.4, "unit": "C"}`. c. Start the **broker first** (`mosquitto -v`), then the **subscribers** (actuator, Node-RED flow), then the **sensor publisher**. A normal (non-retained) message published before anyone subscribes is not delivered later, and clients cannot connect without a broker. d. `mosquitto_pub` =

MQTT publisher client; `-h localhost` = broker on this computer; `-t "greenhouse/zone1/fan/command"` = the topic; `-m "FAN ON"` = the message payload. e. **No change** is needed. The sensor publishes only to the **broker and topic**; it does not know who the subscribers are or how many there are. The broker delivers each message to **every matching subscriber**, so the new dashboard just subscribes. This is decoupling: producers and consumers are independent, which makes the system easy to extend and scale (the week 6 multiple-subscriber experiment).

18. CoAP: lightweight request–response application protocol for constrained devices; expresses "GET /temperature" (resource-oriented, compact binary). UDP: lightweight transport with ports, no connection setup. IPv6: source and destination addressing across networks; huge address space. 6LoWPAN: adaptation layer, header compression (40 → 2–7 bytes), fragmentation/reassembly, mesh routing. IEEE 802.15.4: low-rate, low-power wireless link. Border router: connects the 6LoWPAN mesh to the IPv6 Internet; beyond it the packet is ordinary IPv6. *Why:* the sensor is battery-powered with small memory and tiny messages; TCP and HTTP carry more overhead (handshake, text headers) and Wi-Fi uses more power.

Part A

Week 1: 1 B · **2** D · **3** A · **4** C · **5** B · **6** A · **7** D · **8** C · **9** A · **10** B · **11** D · **12** C · **13** A · **14** B · **15** D · **16** C · **17** B · **18** A · **19** D · **20** C · **21** A · **22** B · **23** D · **24** C · **25** B

Weeks 2–3: 26 C · **27** A · **28** D · **29** B · **30** C · **31** A · **32** D · **33** B · **34** C · **35** A · **36** B · **37** D · **38** C · **39** A · **40** B · **41** D · **42** C · **43** A · **44** B · **45** D · **46** C · **47** A · **48** B · **49** D · **50** C

Week 4: 51 A · **52** C · **53** B · **54** D · **55** A · **56** C · **57** B · **58** D · **59** A · **60** C · **61** B · **62** D · **63** C · **64** A · **65** D · **66** B · **67** A · **68** C · **69** D · **70** B · **71** A · **72** C · **73** D · **74** B · **75** A · **76** C · **77** B · **78** D · **79** A · **80** C · **81** D · **82** B · **83** A · **84** C · **85** B

Week 5: 86 C · **87** A · **88** D · **89** B · **90** C · **91** A · **92** D · **93** B · **94** C · **95** A · **96** D · **97** B · **98** A · **99** C · **100** D · **101** B · **102** A · **103** C · **104** D · **105** B · **106** A · **107** C · **108** D · **109** B · **110** A · **111** C · **112** D · **113** B · **114** A · **115** C · **116** D · **117** B · **118** A · **119** C · **120** D · **121** B · **122** A · **123** C · **124** D · **125** B

Week 6: 126 B · **127** D · **128** A · **129** C · **130** B · **131** A · **132** D · **133** C · **134** B · **135** A · **136** D · **137** C · **138** B · **139** A · **140** D · **141** C · **142** B · **143** A · **144** D · **145** C · **146** B · **147** A · **148** D · **149** C · **150** B · **151** A · **152** D · **153** C · **154** B · **155** A

Part B — True / False

#		Correction / reason
1	F	IoT is about **using data from physical things**.
2	T	
3	F	One year of history belongs in the **cloud**.
4	T	
5	T	The physical–digital loop.
6	F	Devices differ in formats, hardware, OS and communication.
7	F	It needs three readings; before that it is `None` ("Waiting").
8	F	10 °C is valid but TOO_WARM in the cold chain.
9	F	`json.loads()` does that; `json.dumps()` turns a dictionary into JSON text.
10	T	No extra spaces.
11	F	Real range depends on power, antenna, obstacles, interference, environment.
12	F	Bandwidth = maximum; data rate = actually transferred.
13	T	
14	T	Less radio-on time.
15	F	"Technically possible ≠ architecturally appropriate"; match the requirements.
16	T	
17	T	Relay burden.
18	F	RFID uses radio waves; no line of sight needed.
19	T	
20	F	LoRa is **non-cellular**, unlicensed; NB-IoT and LTE-M are cellular.
21	T	
22	T	Trackers vs meters.
23	F	The trade-off triangle.
24	T	
25	T	
26	F	Private addresses are not directly routable on the public Internet (NAT).
27	F	IPv6 solves the address shortage only; 6LoWPAN adapts it for constrained links.
28	T	
29	F	Transport = **process-to-process**; network = host-to-host.
30	F	UDP has no ordering.
31	T	Duplicate handling is one of TCP's four ideas.
32	F	HTTP is request–response; MQTT is publish–subscribe.
33	T	
34	F	404 = not found; 500 = server error.
35	F	Polling creates repeated requests, unnecessary traffic and poor scaling.

#		Correction / reason
36	F	It knows only the broker, topic and payload.
37	F	The payload is application-defined bytes.
38	F	`+` = exactly one level; `#` = all remaining levels.
39	T	`#` matches zero or more remaining levels.
40	T	At least once.
41	F	QoS 2 = PUBLISH → PUBREC → PUBREL → PUBCOMP.
42	F	Only the latest retained message per topic.
43	F	Only on an **unexpected** disconnect.
44	T	They are not mutually exclusive.
45	F	UDP needs no `connect()`; it uses `sendto()`.
46	T	Hence `encode()` / `decode()`.
47	T	
48	F	MQTT uses TCP and adds messaging on top.
49	F	Start the subscriber **first**; non-retained messages sent earlier are not delivered later.
50	F	It subscribes to the **command** topic.

Part C

C1. 1. 1/30 = **3.33 %** · 2. 0.5/60 = **0.83 %** · 3. 2/600 = **0.33 %** · 4. 1/900 = **0.11 %** · 5. 5/3600 = **0.14 %** · 6. 1440 s = 0.4 h × 20 mA = **8 mAh/day** · 7. 288 s = 0.08 h × 30 mA = **2.4 mAh/day** · 8. 72 s = 0.02 h × 50 mA = **1 mAh/day** · 9. 144 × 2 s = 288 s = 0.08 h × 45 mA = **3.6 mAh/day** · 10. 2400 ÷ 12 = **200 days**; 2400 ÷ 0.20 = **12,000 days** (≈ 33 years). Real life is much shorter because energy also goes to sensing, the processor, radio start-up, listening, retransmissions, sleep current and battery self-discharge.

C2. 1. 86,400 ÷ 10 = **8,640** · 2. 50 × 1,440 = **72,000** · 3. **86,400,000** · 4. 200 × 1,440 = **288,000 messages**; × 60 bytes = **17,280,000 bytes ≈ 17.3 MB/day** · 5. Before: 100 × 86,400 = **8,640,000**; after: 100 × 1,440 = **144,000**; **60 times** less traffic.

C3. 1. `11000000.10101000.00000001.00011001` · 2. `00001010.00000000.00000000.00000001` · 3. `10101100.00010000.00000101.11001000` · 4. `2001:db8::1` · 5. `fe80::204:61ff:fe9d:f156` · 6. `2001:db8:abcd:12::25` · 7. `2001:0db8:0000:0000:0000:ff00:0042:8329` · 8. The message to **203.0.113.10** (outside the local network); `192.168.1.50` is local.

C4. 1. Waiting, Waiting, **25.0**, **25.33** · 2. Waiting, Waiting, **30**, **40**, **40**: the single 60 spike appears as a smaller bump (40) spread over several outputs · 3. Waiting, Waiting, **20**, **30**, **40** (the average lags behind a rising trend) · 4. Only the fifth output exists: (30+30+30+60+30) ÷ 5 = **36**. The **larger window** reduces the spike more (+6 instead of +10), but reacts more slowly.

C5. 1. T1, T3 · 2. T1, T2 · 3. T2, T4 · 4. T1, T2, T3, T4, T5, T7 · 5. T4, T6 · 6. T4, T5 · 7. T1 · 8. **None** (`+` is one level; these topics have two levels after `farm`) · 9. **All** (T1–T7) · 10. T6 · 11. **None** (T5 has two levels after `greenhouse`) · 12. **None** (`+` needs exactly one level; `farm` has zero and the others have three)

C6. 1. **Ethernet** (PoE): video needs high data rate, fixed, cabling exists, stable · 2. **BLE**: short range, small data, very low power, phone nearby · 3. **LoRa**: kilometres of range, tiny data, very low power, private gateway · 4. **NB-IoT**: static, low data, good indoor/underground coverage, operator network · 5. **LTE-M / cellular**: wide-area mobility · 6. **RFID**: many tags at once, no line of sight · 7. **NFC**: centimetre range, contactless · 8. **Wi-Fi**: high throughput, AP already at home, mains power · 9. **IEEE 802.15.4 (Zigbee) mesh**: low power, small periodic data, many nodes · 10. **Satellite**: regional/global coverage · 11. **Ethernet**: stable, low latency, fixed · 12. **LTE-M**: small, mobile, battery-powered, city-wide

C7. 1. Connectivity/link · 2. Network · 3. Transport · 4. Application · 5. Adaptation · 6. Transport (identifies the MQTT process) · 7. Data representation · 8. Network (IPv4) · 9. Transport (UDP) · 10. Application · 11. Connectivity/link · 12. Application

C8. 1. An **IPv4 TCP** socket · 2. Sends one **UDP datagram** with the bytes `Hello` to port 5000 on this computer, without any connection · 3. Sockets transmit **bytes**, not strings: use `"Hello".encode()` or `b"Hello"` · 4. The string `'{"t": 22.5}'`, type `str` · 5. The **payload size in bytes** · 6. `data` = the received bytes; `addr` = the sender's `(IP, port)` · 7. **FAN OFF** (30.0 is not > 30) · 8. Converts the payload **bytes into text** · 9. **At least once**; the broker answers with **PUBACK** (duplicates possible) · 10. `lab/room1/temp 21`

Part D — Model answers

1. A smartwatch alone senses and displays heart rate: it is an **IoT device**. A complete **IoT system** also includes communication (BLE to the phone, Internet), processing (cloud platform), an application (health app) and a user or action (doctor alert): smartwatch → smartphone → Internet → cloud → health application → user.
2. Latency → faster local processing; bandwidth → filter locally, send only important data; network dependency → local processing, local storage, retry; reliability → health checks, backup sensors/gateways, buffering, error handling; security → protect all layers (device, network, cloud, application); privacy → keep data local when possible (edge computing).
3. Data from sensors vs typed by humans; physical + digital vs mostly digital; many devices vs a few computers; systems may act automatically vs user-initiated actions; distributed vs centralized computing.
4. The timestamp shows when the value was measured, how fast it is changing and whether it is current. Previous readings show trends, let the system judge whether a value is abnormal compared with normal behaviour, and allow smoothing (moving average).
5. **Validation** checks whether a value is physically possible (temperature −20 to 50 °C → VALID/INVALID): it checks the sensor. **Status** checks whether the value is acceptable for the cargo (2–8 °C → NORMAL, below → TOO_COLD, above → TOO_WARM): it checks the situation. 10 °C is VALID but TOO_WARM.
6. A dictionary is a Python in-memory structure; a JSON string is a standard text representation that any system can read. **Serialization** (`json.dumps`) converts structured data into a form suitable for transmission or storage. Payload size matters because more bytes mean more radio-on time, energy and bandwidth; compact JSON is smaller.
7. **Connectivity** is how devices get connected (the wired or wireless link: Wi-Fi, BLE, LoRa…), the road. A **communication protocol** is how data is exchanged once the link exists (IP, TCP/UDP, HTTP/CoAP/MQTT), the rules of the road.
8. Duty cycle = active radio time ÷ total time. Transmit/receive need high power and sleep needs very little, so the less time the radio is on, the less energy is used and the longer the battery lasts (1 s/min = 1.67 % vs 1 s/h = 0.028 %, 60× energy difference).
9. **Wi-Fi**: high data rate, local, medium–high power, needs an AP (camera, appliance). **BLE**: short range, very low power, phone/gateway (wearable). **LoRa**: long range, very low power and data rate, private gateway (farm sensor). **NB-IoT**: cellular LPWAN, static low-data devices, good indoor coverage (meter). **LTE-M**: cellular, higher data rate, mobility (tracker, scooter).
10. **Star**: all devices connect through one central node (Wi-Fi AP); simple and easy to manage but depends on the centre, and every device must reach it. **Mesh**: nodes connect through neighbours; extended coverage and alternative paths, but more complexity and a relay burden that costs energy.
11. **TCP**: sequence/order, acknowledgment (ACK = next expected byte), retransmission, duplicate handling; connection set up by SYN, SYN-ACK, ACK. **UDP**: connectionless, unreliable delivery, no ordering, no flow control.

12. Both are request–response and resource-oriented. HTTP usually runs over TCP with text messages and suits web/cloud APIs; CoAP usually runs over UDP with compact binary messages and suits constrained devices and networks.

13. **Polling** = repeatedly asking every device for new data even when nothing changes, creating repeated requests, unnecessary traffic and poor scaling. In **publish–subscribe** the sensor publishes to a topic on a broker only when it has data; the broker forwards it to every subscriber. Sensors and applications are decoupled and new applications just subscribe.

14. A **retained message** is the latest message on a topic, kept by the broker and delivered immediately to new subscribers (latest state, not history). A **Last Will** is defined when a client connects and published by the broker only if that client disconnects unexpectedly (e.g. `status = offline`).

15. Broker `mosquitto -v`; `mqtt_sensor.py` publishes JSON temperature to `factory/machine01/temperature` (**sense + communicate**); `edge_processor.py` subscribes to that topic, parses the JSON and applies temperature > 30 °C (**process**), then publishes `FAN ON` / `FAN OFF` to `factory/machine01/fan/command`; `mqtt_actuator.py` subscribes to the command topic and "switches" the fan (**act**). Node-RED can replace the edge processor with MQTT In → JSON → Switch → Change → MQTT Out.